Metody tworzenia aplikacji bazodanowych

Sprawozdanie projektu Service Desk

Stanisław Dutkiewicz 329076

10 Grudnia 2025

1 Wstęp

1.1 Cel projektu

Celem projektu jest przygotowanie prostego systemu do obsługi zgłoszeń IT, tworzonego w ramach przedmiotu Metody tworzenia aplikacji bazodanowych. Aplikacja pozwala pracownikom zgłaszać problemy, a technikom zarządzać zgłoszeniami i kontrolować ich status. Taki system może być wykorzystywany wewnętrznie w firmie lub na uczelni jako podstawowe narzędzie do rejestrowania i śledzenia zgłoszeń.

2 Zakres funkcjonalny systemu

2.1 Opis głównych funkcji (F1–F7)

Zakres funkcjonalny systemu Service Desk został podzielony na siedem głównych funkcji:

Tabela funkcji F1–F7

Kod	Nazwa	Opis
F1	Rejestracja zgłoszeń	Umożliwia utworzenie zgłoszenia z tytułem, opisem, priorytetem i działem zgłaszającym.
F2	Obsługa cyklu życia zgłoszenia	Przypisywanie zgłoszeń do techników i zmiana statusu (OPEN, IN_PROGRESS, RESOLVED, CLOSED).
F3	Przeglądanie i filtrowanie zgłoszeń	Filtrowanie po statusie, techniku, dziale oraz widoki „Moje zgłoszenia” i „Przypisane do mnie”.
F4	Komentarze i historia	Dodawanie komentarzy do zgłoszeń oraz podgląd historii działań.
F5	Raport liczby zgłoszeń	Zestawienie liczby zgłoszeń otwartych i zamkniętych wg działu lub technika.
F6	Raport czasu rozwiązania	Raport średniego czasu rozwiązania na podstawie dat utworzenia i zamknięcia.
F7	Zarządzanie słownikami i użytkownikami	CRUD dla użytkowników, techników, działów i kategorii problemów.

2.2 Role użytkowników i scenariusze użycia

2.2.1 Rola „Użytkownik”

Użytkownik to pracownik organizacji, który zgłasza problemy techniczne. Może:

- logować się do systemu,
- przeglądać własne zgłoszenia,
- dodawać nowe zgłoszenia (F1),
- śledzić status swoich zgłoszeń (F3),
- dodawać komentarze do swoich zgłoszeń (F4).

2.2.2 Rola „Technik IT”

Technik IT odpowiada za obsługę zgłoszeń. Może:

- logować się do systemu,
- przeglądać zgłoszenia przypisane do siebie (F3),
- zmieniać status zgłoszeń (np. OPEN → IN_PROGRESS → RESOLVED/CLOSED) (F2),
- dodawać komentarze i opisy rozwiązań (F4).

2.2.3 Rola „Administrator"

Administrator zarządza konfiguracją systemu. Może:

- przeglądać wszystkie zgłoszenia bez ograniczeń,
- zarządzać użytkownikami i technikami – operacje CRUD poprzez REST API (F7),
- zarządzać działami i kategoriami problemów – operacje CRUD (F7),
- korzystać z raportów liczby zgłoszeń oraz średniego czasu rozwiązania (F5, F6).

Uwaga: Obecnie endpointy CRUD nie sprawdzają ról użytkowników, ponieważ system nie ma jeszcze wprowadzonego mechanizmu autoryzacji. Gdyby projekt był dalej rozwijany, należałoby dodać kontrolę dostępu w kontrolerach, tak aby np. operacje administracyjne były dostępne tylko dla wybranych użytkowników.

3 Model danych i projekt bazy

3.1 Przegląd encji i tabel

Model danych systemu Service Desk opiera się na relacyjnej bazie danych. Poniżej przedstawiono główne tabele (encji) oraz ich przeznaczenie.

Rdzeń bazy danych – spełniający wymaganie co najmniej pięciu tabel „nie-uprawnieniowych" – tworzą:

- `departments` – słownik działów,
- `categories` – słownik kategorii problemów,
- `tickets` – zgłoszenia serwisowe,
- `comments` – komentarze i historia,
- `technicians` – technicy IT.

Tabele dodatkowe:
`users` – encja reprezentująca użytkowników (zgłaszających problemy i autorów komentarzy). Jest to tabela biznesowa niezbędna do funkcjonowania systemu, powiązana zarówno z ticketami, technicians, jak i comments.
`accounts` – tabela uwierzytelniania i zarządzania rolami użytkowników, powiązana z users i technicians.

3.1.1 Tabela `users`

Tabela `users` przechowuje dane użytkowników zgłaszających problemy:

- `id` – klucz główny (identyfikator użytkownika),
- `first_name`, `last_name` – imię i nazwisko,
- `email` – adres e-mail (unikalny),
- `department_id` – odwołanie do tabeli `departments`.

3.1.2 Tabela `technicians`

Tabela `technicians` przechowuje informacje o technikach IT obsługujących zgłoszenia:

- `id` – klucz główny,
- `first_name`, `last_name` – imię i nazwisko technika,
- `email` – adres e-mail technika (unikalny),
- `department_id` – dział, w którym pracuje technik.

3.1.3 Tabela `departments`

Tabela `departments` jest słownikiem działów w organizacji:

- `id` – klucz główny,
- `name` – nazwa działu (np. HR, IT, Maintenance, Cargo).

3.1.4 Tabela `categories`

Tabela `categories` przechowuje kategorie problemów:

- `id` – klucz główny,
- `name` – nazwa kategorii (np. Sprzęt, Oprogramowanie, Konto).

3.1.5 Tabela `tickets`

Tabela `tickets` reprezentuje zgłoszenia serwisowe:

- `id` – klucz główny,
- `title` – tytuł zgłoszenia,
- `description` – opis problemu,
- `priority` – priorytet (LOW, MEDIUM, HIGH, CRITICAL),
- `status` – status zgłoszenia (OPEN, IN_PROGRESS, RESOLVED, CLOSED),
- `created_at` – data utworzenia,
- `closed_at` – data zamknięcia (opcjonalna),
- `reporter_id` – odwołanie do użytkownika zgłaszającego (tabela `users`),
- `technician_id` – odwołanie do technika obsługującego (tabela `technicians`),
- `category_id` – odwołanie do kategorii problemu (tabela `categories`).

3.1.6 Tabela `comments`

Tabela `comments` przechowuje komentarze i historię działań przy zgłoszeniach:

- `id` – klucz główny,
- `ticket_id` – odwołanie do zgłoszenia (tabela `tickets`),
- `author_id` – odwołanie do autora komentarza (tabela `users`),
- `content` – treść komentarza,
- `created_at` – data dodania komentarza.

3.1.7 Tabela `accounts`

Tabela `accounts` służy do uwierzytelniania i przechowywania ról użytkowników:

- `id` – klucz główny,
- `login` – login użytkownika (unikalny),
- `password` – hasło (**dla celów dydaktycznych przechowywane jawnie** – w środowisku produkcyjnym wymaga hashowania, np. BCrypt),
- `role` – rola w systemie (`admin`, `technician`, `user`),
- `user_id` – powiązanie z rekordem w tabeli `users` (dla zwykłych użytkowników i kont technicznych; umożliwia technikowi dodawanie komentarzy),
- `technician_id` – powiązanie z rekordem w tabeli `technicians` (dla kont technicznych; umożliwia obsługę zgłoszeń).

Uwaga: Technicy mają podwójną tożsamość – zarówno `user_id` jak i `technician_id` – aby mogli zarówno obsługiwać zgłoszenia, jak i dodawać komentarze jako autorzy.

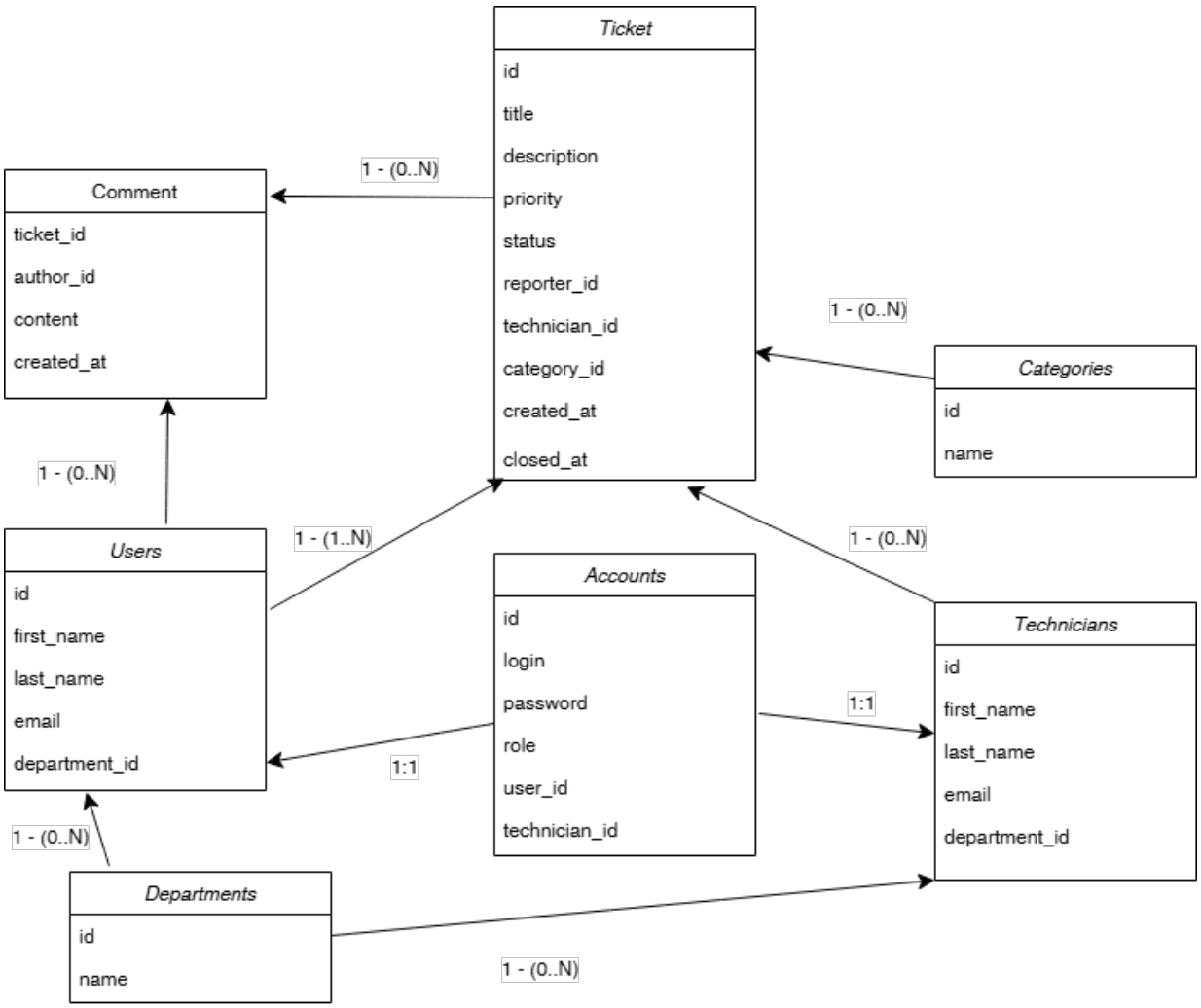
3.2 Relacje pomiędzy tabelami

Między tabelami występują następujące kluczowe relacje:

- jeden dział (`departments`) może mieć wielu użytkowników (`users`) oraz wielu techników (`technicians`) – relacje typu 1:N,
- jeden użytkownik (`users`) może utworzyć wiele zgłoszeń (`tickets`) – relacja 1:N,
- jeden technik (`technicians`) może obsługiwać wiele zgłoszeń (`tickets`) – relacja 1:N,
- jedno zgłoszenie (`tickets`) może mieć wiele komentarzy (`comments`) – relacja 1:N,
- każdy komentarz jest powiązany z jednym zgłoszeniem i jednym autorem (użytkownikiem) – relacje N:1 do `tickets` i `users`,
- tabela `accounts` łączy loginy z użytkownikami i technikami, umożliwiając powiązanie konta z odpowiednim rekordem w `users` i/lub `technicians`.

3.3 Diagram ERD systemu Service Desk

W niniejszym podrozdziale można umieścić graficzny diagram ERD przedstawiający opisane powyżej tabele i relacje. Przykładowo:



Rysunek 1: Diagram ERD systemu Service Desk

4 Architektura techniczna systemu

4.1 Zastosowane technologie

System został zaprojektowany jako aplikacja webowa w architekturze trójwarstwowej (przeglądarka WWW – serwer Spring Boot – relacyjna baza danych). Do implementacji użyto:

Backend:

- Java 21,
- Spring Boot 3.3.4,
- Spring Data JPA (ORM),
- H2 Database (środowisko dev) / PostgreSQL (produkcja).

Frontend:

- Vanilla HTML5, CSS3, JavaScript (bez frameworków typu React/Angular),
- przechowywanie stanu użytkownika w zmiennej JavaScript `currentUser`,
- dynamiczne renderowanie interfejsu opartej na rolach użytkownika.

Podsumowanie głównych endpointów REST API

Ścieżka	Metoda	Wejście (najważniejsze pola)	Cel / powiązana funkcja
`/api/tickets`	POST	`title`, `description`, `priority`, `reporterId`, `technicianId`, `categoryId`	Utworzenie nowego zgłoszenia (F1). Zwraca obiekt zgłoszenia z nadanym `id` i statusem `OPEN`.
`/api/tickets`	GET	parametry opcjonalne: `status`, `technicianId`, `departmentId`	Lista zgłoszeń z możliwością filtrowania (F3).
`/api/tickets/my`	GET	`userId`	Lista zgłoszeń utworzonych przez danego użytkownika (F3).
`/api/tickets/assigned`	GET	`technicianId`	Lista zgłoszeń przypisanych do technika (F2, F3).
`/api/tickets/{id}/status/{status}`	POST	brak ciała lub proste DTO	Zmiana statusu zgłoszenia, uzupełnienie daty zamknięcia przy RESOLVED/CLOSED (F2).
`/api/comments`	POST	`ticketId`, `authorId`, `content`	Dodanie komentarza do zgłoszenia (F4).
`/api/comments/ticket/{ticketId}`	GET	`ticketId`	Lista komentarzy powiązanych ze zgłoszeniem (F4).
`/api/reports/tickets-count`	GET	`groupBy` = `department` lub `technician`	Raport liczby zgłoszeń wg działu/technika (F5).
`/api/reports/avg-resolution-time`	GET	–	Raport średniego czasu rozwiązania zgłoszeń (F6).
`/api/login`	POST	`login`, `password`	Logowanie i pobranie roli, `userId` / `technicianId` (panel użytkownika/technika).

4.2 Architektura trójwarstwowa

System został zaprojektowany jako klasyczna aplikacja webowa w architekturze trójwarstwowej: warstwa prezentacji (przeglądarka WWW), warstwa logiki aplikacji (serwer Spring Boot z REST API) oraz warstwa danych (relacyjna baza H2/PostgreSQL).

Wymiana danych: Między warstwą frontendu a backendem odbywa się poprzez REST API z formatem JSON. Frontend wysyła żądania HTTP (GET, POST, PUT, DELETE) do endpointów backendu, a backend zwraca odpowiedzi w formacie JSON wraz z odpowiednimi kodami HTTP (200 OK, 201 Created, 4xx dla błędów).

4.2.1 Warstwa prezentacji (frontend webowy)

Warstwa prezentacji obejmuje:

- ekran logowania,
- panel pracownika (przegląd własnych zgłoszeń oraz formularz nowego zgłoszenia),
- panel technika IT (lista przypisanych zgłoszeń, zmiana statusów, dodawanie komentarzy).

Widoki są zaimplementowane jako strony HTML z prostym JavaScriptem wywołującym endpointy REST API.

4.2.2 Warstwa logiki aplikacji (REST API, serwisy)

Warstwa logiki aplikacji jest zrealizowana jako zestaw kontrolerów REST i serwisów Spring:

- kontrolery `/api/tickets`, `/api/comments`, `/api/users`, `/api/departments`, `/api/categories`, `/api/reports` oraz `/api/login`,
- serwisy odpowiedzialne za obsługę zgłoszeń, komentarzy, raportów oraz uwierzytelniania.

Warstwa ta realizuje reguły biznesowe systemu, m.in. zmianę statusów zgłoszeń, przypisywanie techników, obliczanie raportów i weryfikację danych wejściowych.

4.2.3 Warstwa danych (baza H2/PostgreSQL)

Warstwa danych obejmuje:

- encje JPA odwzorowujące tabele bazy danych,
- repozytoria `JpaRepository` odpowiedzialne za operacje CRUD,
- konfigurację bazy H2 (środowisko testowe) z możliwością przełączenia na PostgreSQL.

Struktura bazy danych jest inicjalizowana przy starcie aplikacji na podstawie encji oraz pliku `data.sql` zawierającego dane testowe.

4.3 Diagram architektury systemu

W tej części można zamieścić uproszczony diagram architektury, przedstawiający przepływ danych pomiędzy warstwami:

5 Interfejsy i realizacja funkcjonalności

5.1 Interfejs REST API

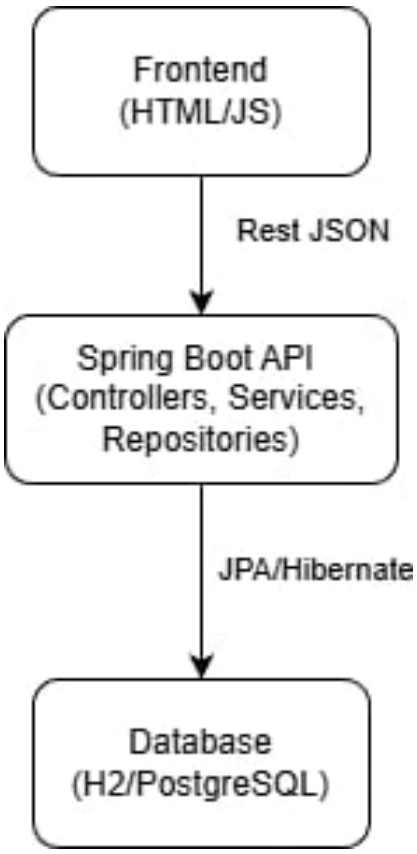
Backend systemu Service Desk udostępnia zestaw endpointów REST API pod adresem bazowym:

`http://localhost:8080/api`

5.1.1 Konfiguracja CORS

System jest skonfigurowany z włączoną obsługą Cross-Origin Resource Sharing (CORS), co umożliwia żądania ze stron:

- `http://localhost:3000` – alternatywny serwer deweloperski,
- `http://localhost:8080` – ta sama domena,
- `file://` – lokalne pliki HTML (dla celów testowania).



Rysunek 2: Architektura trójwarstwowa systemu Service Desk

5.1.2 Zgłoszenia (`/api/tickets`, F1–F3)

Utworzenie nowego zgłoszenia (F1)

- Metoda: `POST`
- Ścieżka: `/api/tickets`
- Treść żądania (JSON, przykładowo):

```
{
  "title": "Nie działa drukarka",
  "description": "Drukarka w pokoju 204 nie drukuje",
  "reporterId": 1,
  "technicianId": 1,
  "categoryId": 1,
  "priority": "HIGH"
}
```

W odpowiedzi zwracany jest obiekt zgłoszenia z nadanym identyfikatorem, statusem `OPEN` oraz danymi powiązanych encji (użytkownik, technik, kategoria). Przy poprawnym utworzeniu zwracany jest kod statusu HTTP `201 Created`.

Pobranie listy zgłoszeń (F3)

- Metoda: `GET`
- Ścieżka: `/api/tickets`
- Parametry opcjonalne:

- `status` – filtr po statusie (np. `OPEN`, `IN_PROGRESS`),
- `technicianId` – filtr po przypisanym techniku,
- `departmentId` – filtr po dziale zgłaszającego.

Przykładowe wywołania:

```
GET /api/tickets
GET /api/tickets?status=OPEN
GET /api/tickets?technicianId=1
GET /api/tickets?status=IN_PROGRESS&technicianId=1
```

Zgłoszenia użytkownika i technika System udostępnia wyspecjalizowane endpointy dla panelu użytkownika i technika:

- `GET /api/tickets/my?userId={id}` – lista zgłoszeń utworzonych przez danego użytkownika,
- `GET /api/tickets/assigned?technicianId={id}` – zgłoszenia przypisane do danego technika.

Szczegóły zgłoszenia

- Metoda: `GET`
- Ścieżka: `/api/tickets/{id}`
- Opis: zwraca pełne informacje o pojedynczym zgłoszeniu, łącznie z powiązanymi encjami.

Przypisanie zgłoszenia do technika (F2)

- Metoda: `POST`
- Ścieżka: `/api/tickets/{id}/assign`
- Treść żądania:

```
{
  "technicianId": 2,
  "status": "IN_PROGRESS"
}
```

Zmiana statusu zgłoszenia (F2)

- Metoda: `POST`
- Ścieżka: `/api/tickets/{id}/status/{status}`
- Opis: zmiana statusu zgłoszenia, przy przejściu do `RESOLVED` lub `CLOSED` uzupełniana jest data zamknięcia.

5.1.3 Komentarze (`/api/comments`, F4)

Dodanie komentarza do zgłoszenia

- Metoda: `POST`
- Ścieżka: `/api/comments`
- Treść żądania:

```
{
  "ticketId": 100,
  "authorId": 4,
  "content": "Sprawdzę sterowniki"
}
```

Pobranie komentarzy dla zgłoszenia

- Metoda: `GET`
- Ścieżka: `/api/comments/ticket/{ticketId}`
- Opis: zwraca listę komentarzy wraz z informacjami o autorze i dacie dodania.

5.1.4 Słowniki i użytkownicy (`/api/users`, `/api/technicians`, `/api/departments`, `/api/categories`, F7)

Dla zarządzania słownikami i listą użytkowników udostępniono proste operacje CRUD, m.in.:

- `GET /api/users` – lista użytkowników,
- `POST /api/users` – dodanie użytkownika,
- `PUT /api/users/{id}` – modyfikacja użytkownika,
- `DELETE /api/users/{id}` – usunięcie użytkownika,
- `GET /api/technicians` – lista techników,
- `GET /api/departments` – lista działów,
- `GET /api/categories` – lista kategorii.

5.1.5 Raporty (`/api/reports/...`, F5–F6)

Raport liczby zgłoszeń (F5)

- Metoda: `GET`
- Ścieżka: `/api/reports/tickets-count`
- Parametr: `groupBy` – `department` lub `technician`.

Przykład:

```
GET /api/reports/tickets-count?groupBy=department
```

Odpowiedź zwraca listę obiektów zawierających nazwę grupy oraz liczbę zgłoszeń otwartych i zamkniętych.

Raport średniego czasu rozwiązania (F6)

- Metoda: `GET`
- Ścieżka: `/api/reports/avg-resolution-time`
- Opis: zwraca średni czas rozwiązania zgłoszeń w godzinach oraz liczbę uwzględnionych rekordów.

5.2 Widoki użytkownika (frontend)

Warstwa prezentacji została zrealizowana jako lekkie strony HTML z JavaScriptem wykorzystującym powyższe endpointy REST.

5.2.1 Ekran logowania

Ekran logowania udostępnia interfejs do wyboru roli użytkownika z trzema przyciskami:

- Administrator,
- Technik IT,
- Użytkownik.

Po kliknięciu przycisku wybranej roli JavaScript bezpośrednio ustawia zmienną `currentUser.role` na stronie przeglądarki. Informacja o roli jest przechowywana w obiekcie `currentUser` w pamięci aplikacji. W zależności od wybranej roli użytkownik jest przekierowywany do:

- panelu pracownika dla roli `user`,
- panelu technika dla roli `technician`,
- widoku administracyjnego dla roli `admin`.

Architektura logowania: System implementuje endpoint `POST /api/login` (AuthController) z serwisem AuthService na poziomie backendu dla przyszłych integracji i produkcyjnego systemu uwierzytelniania. W obecnej wersji demonstracyjnej (deweloperskiej) frontend używa lokalnego mechanizmu wyboru roli bez komunikacji z API, aby umożliwić szybkie testowanie wszystkich scenariuszy funkcjonalności.

Na rysunku 3 przedstawiono przykładowy wygląd ekranu logowania.

5.2.2 Panel pracownika

Panel pracownika składa się z dwóch głównych części:

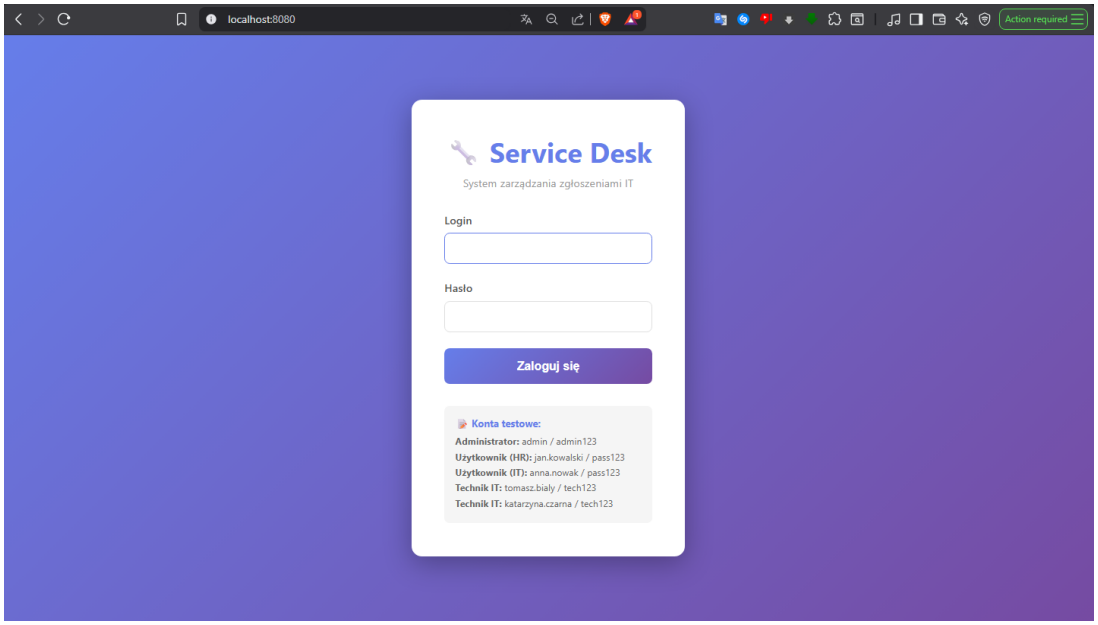
- widoku listy zgłoszeń danego użytkownika,
- formularza tworzenia nowego zgłoszenia.

Lista zgłoszeń pracownika Zakładka **„Moje zgłoszenia”** prezentuje listę wszystkich zgłoszeń utworzonych przez zalogowanego pracownika. Dane są pobierane z endpointu:

```
GET /api/tickets/my?userId={id}
```

Dla każdego zgłoszenia wyświetlane są m.in.:

- tytuł zgłoszenia,
- aktualny status (OPEN, IN_PROGRESS, RESOLVED, CLOSED),

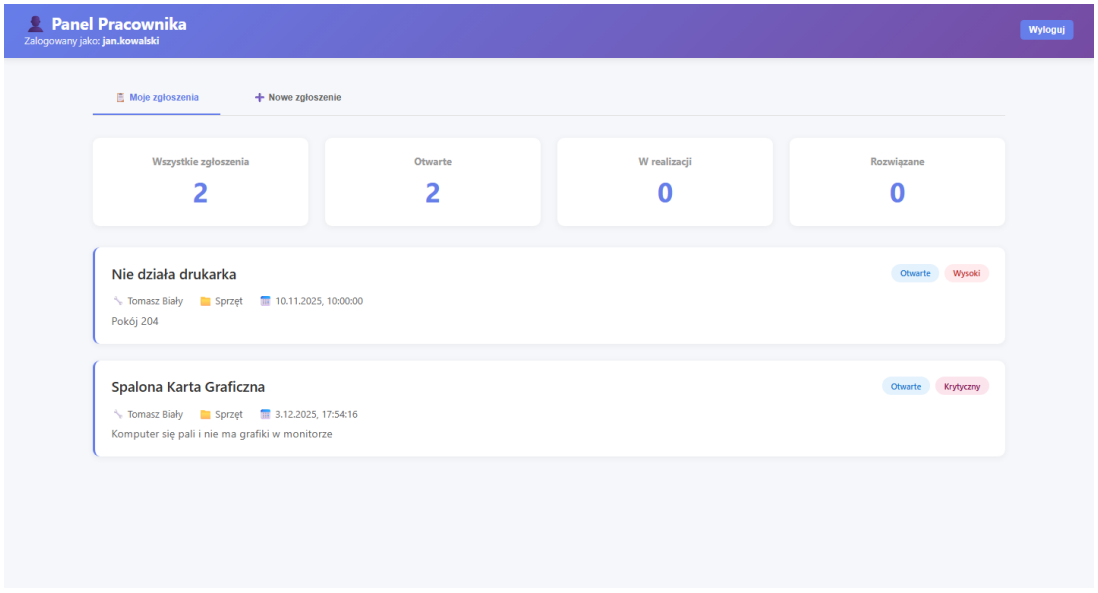


Rysunek 3: Ekran logowania do systemu Service Desk. Użytkownik wybiera rolę, a aplikacja zwraca odpowiedni interfejs (bez komunikacji z API w wersji demo). Funkcje F1–F7.

- priorytet (LOW, MEDIUM, HIGH, CRITICAL),
- przypisany technik (jeśli został wybrany),
- kategoria i data utworzenia.

Kliknięcie w zgłoszenie otwiera okno szczegółów, w którym widoczna jest historia komentarzy oraz dodatkowe informacje.

Na rysunku 4 przedstawiono przykładowy widok listy zgłoszeń zalogowanego pracownika.



Rysunek 4: Panel pracownika – lista własnych zgłoszeń. Dane na liście są pobierane z endpointu `GET /api/tickets/my?userId={id}`, co realizuje funkcję F3 (przeglądanie zgłoszeń zalogowanego użytkownika).

Tworzenie nowego zgłoszenia Zakładka **„Nowe zgłoszenie”** zawiera formularz służący do rejestracji nowego problemu. Formularz odwołuje się do endpointu:

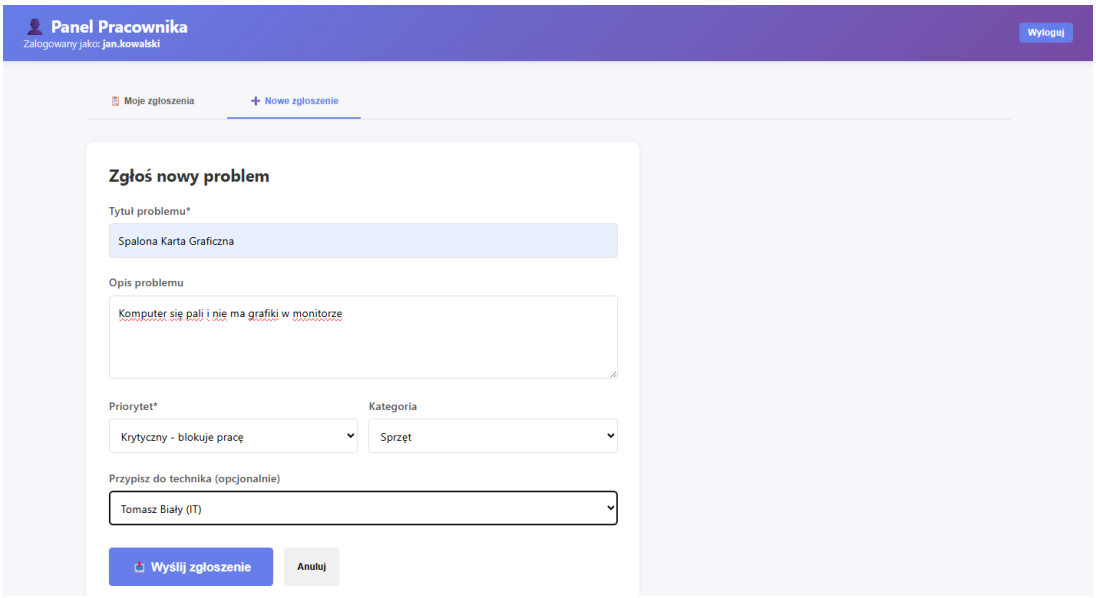
```
POST /api/tickets
```

i pozwala wprowadzić m.in.:

- tytuł problemu (pole wymagane),
- opis problemu,
- priorytet (np. Niski, Średni, Wysoki, Krytyczny),
- kategorię problemu (lista z `GET /api/categories`),
- opcjonalnie przypisanego technika (lista z `GET /api/technicians`).

Po poprawnym zapisaniu zgłoszenia użytkownik otrzymuje informację o sukcesie, a nowe zgłoszenie pojawia się na liście „Moje zgłoszenia".

Rysunek 5 przedstawia przykładowy wygląd formularza tworzenia zgłoszenia.



Rysunek 5: Panel pracownika – formularz tworzenia nowego zgłoszenia. Formularz wysyła żądanie `POST /api/tickets`, tworząc nowe zgłoszenie (F1), które następnie pojawia się na liście „Moje zgłoszenia".

5.2.3 Panel technika IT

Panel technika IT również można podzielić na dwa główne widoki:

- listę zgłoszeń przypisanych do danego technika,
- widok szczegółów zgłoszenia z akcjami rozwiązania problemu.

Lista zgłoszeń technika Główna część panelu prezentuje listę zgłoszeń przypisanych do aktualnie zalogowanego technika. Dane pobierane są z endpointu:

```
GET /api/tickets/assigned?technicianId={id}
```

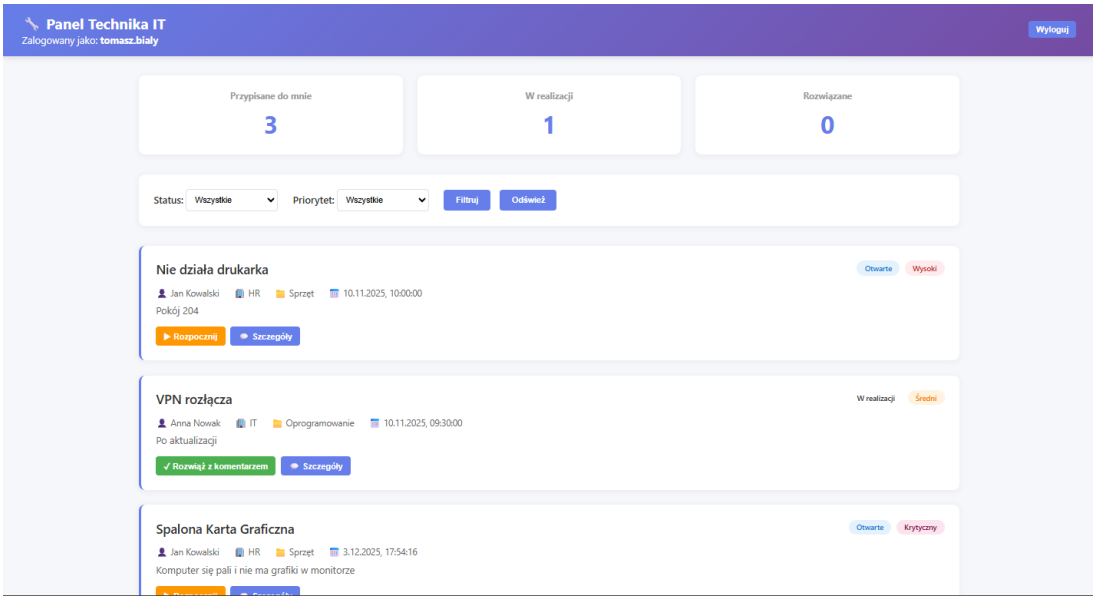
Widok zawiera m.in.:

- tytuł zgłoszenia,
- status i priorytet wyróżnione graficznie (np. kolorowymi etykietami),
- informacje o zgłaszającym i dziale,

- kategorię i datę utworzenia zgłoszenia.

Dodatkowo dostępne są filtry po statusie i priorytecie oraz podstawowe statystyki (liczba zgłoszeń przypisanych, w realizacji, rozwiązanych).

Na rysunku 6 pokazano przykładowy widok listy zgłoszeń dostępny w panelu technika.



Rysunek 6: Panel technika IT – lista przypisanych zgłoszeń. Widok wykorzystuje endpoint `GET /api/tickets/assigned?technicianId={id}`, umożliwiając technikowi przeglądanie przypisanych do niego zgłoszeń (F2, F3).

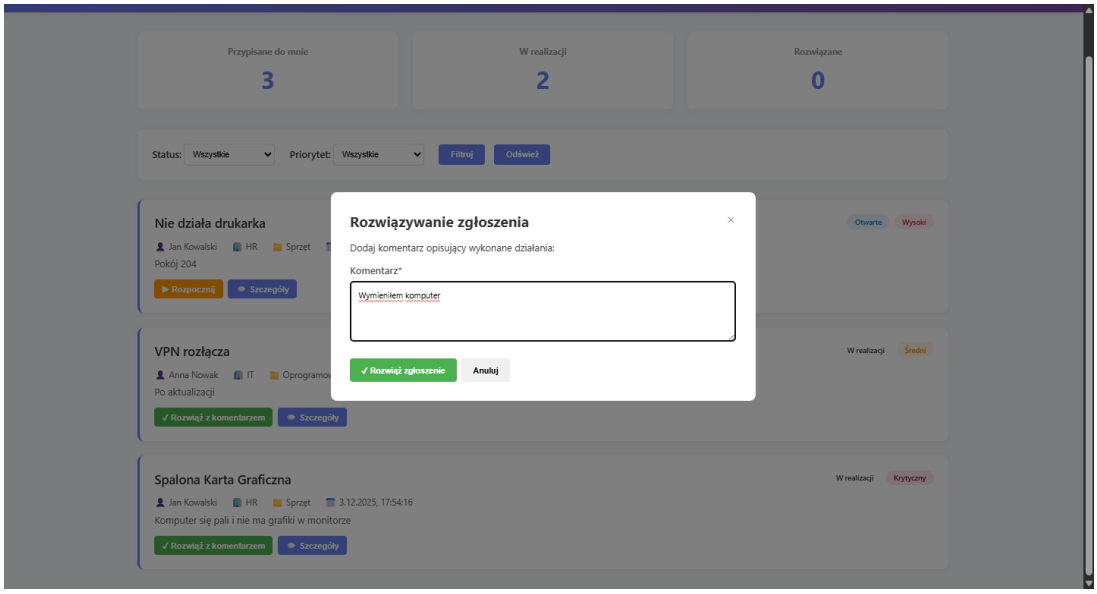
Rozwiązywanie zgłoszenia Po wybraniu konkretnego zgłoszenia technik może przejść do jego szczegółowego widoku. W tym widoku dostępne są m.in.:

- pełne informacje o zgłoszeniu (tytuł, opis, priorytet, status, kategoria, zgłaszający, dział),
- lista wszystkich komentarzy wraz z datą i autorem,
- formularz dodania nowego komentarza,
- przyciski akcji zmieniające status zgłoszenia:
  - rozpoczęcie pracy nad zgłoszeniem (np. OPEN → IN_PROGRESS),
  - oznaczenie zgłoszenia jako rozwiązane z komentarzem (IN_PROGRESS → RESOLVED),
  - zamknięcie zgłoszenia (RESOLVED → CLOSED).

Operacje te realizowane są po stronie backendu poprzez endpointy:

- `POST /api/comments` – dodanie komentarza,
- `POST /api/tickets/{id}/status/{status}` – zmiana statusu zgłoszenia.

Rysunek 7 przedstawia przykładowy widok szczegółów zgłoszenia wraz z możliwością jego rozwiązania i dodawania komentarzy.



Rysunek 7: Panel technika IT – akcja rozwiązania problemu. Zmiana statusu zgłoszenia realizowana jest przez `POST /api/tickets/{id}/status/{status}`, a dodawanie komentarzy przez `POST /api/comments`, co odpowiada funkcjom F2 i F4.

5.2.4 Panel administratora (interfejs webowy)

Panel administratora jest dostępny po zalogowaniu się na konto z rolą `admin` w głównym interfejsie `index.html`. Interfejs służy do przeglądania całości systemu, podstawowego raportowania oraz zarządzania słownikami. Po poprawnym zalogowaniu system wyświetla pasek nawigacyjny zawierający opcje dostępne dla administratora.

Górna część strony zawiera pasek nawigacyjny z informacją o zalogowanym użytkowniku (np. "Zalogowany jako: `admin` (rola: admin)") oraz przyciskiem `Wyloguj`. Przycisk wylogowania czyści dane sesji po stronie przeglądarki i przenosi użytkownika na ekran logowania.

Główny układ panelu obejmuje trzy obszary:

- sekcję **„Raporty systemowe"** z kafelkami liczbowymi, które pokazują m.in. liczbę zgłoszeń otwartych, w realizacji i zamkniętych, z podziałem na działy lub techników,
- tabelę **„Wszystkie zgłoszenia"** z możliwością filtrowania po statusie, dziale i techniku; tabela zawiera kolumny: ID, tytuł, status, priorytet, zgłaszający, dział, technik, kategoria oraz daty utworzenia/zamknięcia,
- panel boczny **„Słowniki i użytkownicy"** z listami użytkowników, techników, działów oraz kategorii problemów (dane słownikowe wykorzystywane w formularzach).

Najważniejsze akcje dostępne w panelu administratora to:

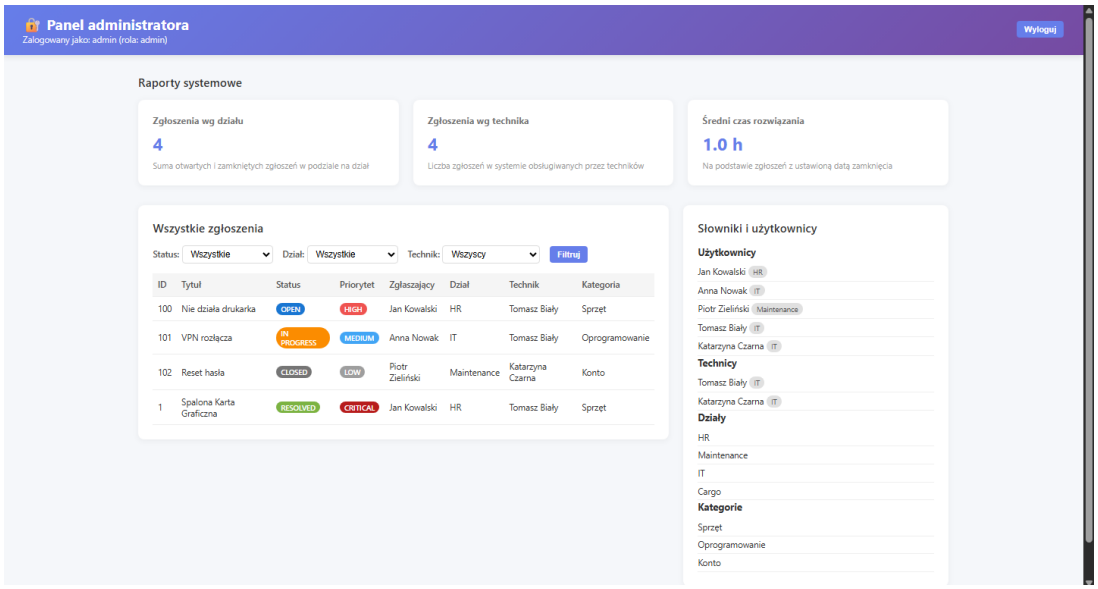
- przeglądanie wszystkich zgłoszeń w systemie z filtrami (tabela),
- podgląd raportów statystycznych (kafle raportowe, widok zbiorczy),
- przegląd i podstawowe zarządzanie słownikami: użytkownicy, technicy, działy, kategorie (operacje CRUD wykonywane przez REST API).

Panel wykorzystuje te same endpointy REST, które zostały opisane w poprzednim podrozdziale, m.in.:

- `GET /api/tickets` – pobranie listy zgłoszeń (opcjonalne parametry: `status`, `departmentId`, `technicianId`),
- `GET /api/reports/tickets-count?groupBy=department` – raport liczby zgłoszeń według działu,

- `GET /api/reports/avg-resolution-time` – średni czas rozwiązania zgłoszeń,
- `GET /api/users`, `GET /api/technicians`, `GET /api/departments`, `GET /api/categories` – dane słownikowe i użytkownicy.

Informacje o aktualnie zalogowanym użytkowniku są przechowywane po stronie przeglądarki w obiekcie `currentUser` (w pamięci aplikacji JavaScript). Przy kliknięciu przycisku wylogowania dane użytkownika są czyszczone i strona przechodzi do ekranu logowania.



Rysunek 8: Panel administratora korzystający z endpointów: `GET /api/tickets` (lista zgłoszeń), `GET /api/reports/tickets-count` (raporty liczbowe), `GET /api/reports/avg-resolution-time` (średni czas rozwiązania) oraz słowników: `GET /api/users`, `GET /api/technicians`, `GET /api/departments`, `GET /api/categories`.

6 Wymagania niefunkcjonalne i techniczne

6.1 Spójność i integralność danych

System wykorzystuje relacyjną bazę danych z jasno zdefiniowanymi kluczami głównymi i obcymi. Relacje pomiędzy tabelami (m.in. `users`–`tickets`, `technicians`–`tickets`, `tickets`–`comments`) są egzekwowane zarówno na poziomie bazy danych, jak i warstwy aplikacji poprzez encje JPA. Dzięki temu zapewniona jest:

- integralność referencyjna (brak „sierocych” rekordów),
- spójność danych pomiędzy zgłoszeniami, użytkownikami, technikami i komentarzami,
- kontrola statusów i priorytetów na poziomie typów wyliczeniowych (enum).

6.2 Walidacja danych

Dane wejściowe są weryfikowane:

- na poziomie encji JPA (adnotacje np. `@NotBlank`, `@Email`),
- w warstwie API poprzez typy DTO i adnotacje walidacyjne,
- dodatkowo na poziomie interfejsu użytkownika (formularze HTML ograniczające puste pola i niepoprawne wartości).

Pozwala to ograniczyć możliwość zapisania do bazy niekompletnych lub sprzecznych danych.

6.3 Bezpieczeństwo i ochrona danych

System przechowuje dane osobowe użytkowników (imiona, nazwiska, e-maile). W zgodzie z wymogami RODO:

- dane są przechowywane w zabezpieczonej bazie danych,
- dostęp do panelu jest chroniony logowaniem (system kont),
- dla przyszłych rozszerzeń rekomenduje się:
  - hashowanie haseł za pomocą BCrypt lub podobnego algorytmu (aktualne przechowywanie jawne jest wyłącznie dla celów dydaktycznych),
  - implementacja Spring Security z kontrolą dostępu opartą na rolach (Role-Based Access Control),
  - szyfrowanie wrażliwych danych (opcjonalne),
  - logowanie dostępu do danych osobowych.

6.4 Konfiguracja środowiska i baza danych

W środowisku deweloperskim system korzysta z wbudowanej bazy H2 w trybie in-memory, skonfigurowanej w trybie zgodności z PostgreSQL. Przy starcie aplikacji:

- Hibernate generuje schemat bazy danych na podstawie encji JPA,
- plik `data.sql` ładuje dane testowe (działy, użytkowników, techników, konta logowania, kategorie, przykładowe zgłoszenia i komentarze).

Docelowo system może zostać przełączony na bazę PostgreSQL poprzez zmianę parametrów połączenia w pliku konfiguracyjnym (`application.yml`) bez modyfikacji logiki aplikacji.

6.5 Wydajność i skalowalność

System został zaprojektowany jako lekka aplikacja webowa przeznaczona dla małej lub średniej liczby użytkowników (np. dział IT w firmie). Zastosowanie Spring Boot oraz JPA umożliwia w przyszłości:

- przeniesienie aplikacji na wydajniejszy serwer aplikacyjny,
- zmianę bazy danych na bardziej wydajną konfigurację produkcyjną,
- optymalizację zapytań i wprowadzenie paginacji list zgłoszeń.

7 Instrukcja uruchomienia systemu

7.1 Wymagania systemowe

Do uruchomienia systemu Service Desk niezbędne są:

- JDK w wersji 21 lub wyższej,
- Apache Maven 3.6+,
- przeglądarka internetowa (Chrome, Firefox, Edge, Safari),
- port 8080 dostępny (domyślny port aplikacji).

7.2 Uruchomienie aplikacji

7.2.1 Etap 1: Budowa projektu Maven

Przejdź do katalogu głównego projektu (folder `api/`):

```
cd api
```

Zbuduj projekt poleceniem:

```
mvn clean install
```

To polecenie:

- pobiera wszystkie zależności (Spring Boot, JPA, itp.),
- kompiluje kod Java,
- kopiuje pliki frontendu z katalogu `../web/` do `target/classes/web/`,
- tworzy plik JAR gotowy do uruchomienia.

7.2.2 Etap 2: Uruchomienie serwera Spring Boot

Po pomyślnej budowie uruchom aplikację:

```
mvn spring-boot:run
```

lub bezpośrednio:

```
java -jar target/classes/../*.jar
```

Serwer powinien uruchomić się na porcie `8080`. W konsoli powinna pojawić się informacja:

```
Tomcat started on port(s): 8080 (http)
```

7.2.3 Etap 3: Dostęp do aplikacji

Otwórz przeglądarkę i przejdź do:

`http://localhost:8080/index.html`

Interfejs REST API jest dostępny pod adresem:

`http://localhost:8080/api`

7.3 Testowanie roli użytkownika

Na ekranie logowania można wybrać jedną z trzech ról:

- **Administrator** – pełny dostęp do systemu, przeglądanie wszystkich zgłoszeń, zarządzanie słownikami,
- **Technik IT** – obsługa przypisanych zgłoszeń, zmiana statusów, dodawanie komentarzy,
- **Użytkownik** – tworzenie nowych zgłoszeń, przeglądanie własnych zgłoszeń, dodawanie komentarzy.

Każda rola otwiera dedykowany panel z odpowiednimi funkcjami.

7.4 Dane testowe

Podczas startu aplikacji automatycznie ładowane są dane testowe z pliku `src/main/resources/data.sql`, zawierające:

- 3 działy (IT, HR, Maintenance),
- 5 użytkowników testowych,
- 2 techników IT,
- 4 kategorie problemów,
- Przykładowe zgłoszenia i komentarze.

7.5 Zatrzymanie aplikacji

Aby zatrzymać serwer, wciśnij kombinację klawiszy `Ctrl+C` w oknie konsoli, w którym uruchomiona jest aplikacja.

7.6 Uruchamianie i środowisko wykonawcze

Aplikacja backendowa wymaga:

- JDK w wersji 21,
- narzędzia Maven do budowy projektu,
- środowiska uruchomieniowego Spring Boot (wbudowany serwer Tomcat).

Domyślnie serwer HTTP nasłuchuje na porcie `8080`, a interfejs REST API jest dostępny pod adresem:

`http://localhost:8080/api`

7.6.1 Konfiguracja bazy danych

System automatycznie uruchamia się z bazą H2 w trybie in-memory (profil `h2`), co wymaga zaledwie Java i Maven. Dane testowe są automatycznie ładowane z pliku `data.sql` podczas startu aplikacji.

Dla środowiska produkcyjnego można przełączyć profil na PostgreSQL poprzez:

```
mvn spring-boot:run -Dspring-boot.run.arguments="-spring.profiles.active=postgres"
```

lub modyfikacją pliku `application.yml` (zmiana `active: h2` na `active: postgres`).

7.6.2 Konfiguracja Maven i pliki statyczne

System buduje się przy pomocy Maven z następującą konfiguracją:

- pliki z `src/main/resources/` (konfiguracja, dane) są kopiowane do `target/classes/`,
- pliki z katalogu `../web/` (HTML, CSS, JavaScript) są kopiowane do `target/classes/web/`,
- WebConfig mapuje ścieżki `/` na `classpath:/web/` i `classpath:/static/`.

Frontend jest dostępny pod adresem:

`http://localhost:8080/index.html`

8 Wnioski i możliwości rozwoju systemu

8.1 Ocena projektu

Zrealizowany system spełnia cele projektu z przedmiotu *Metody tworzenia aplikacji bazodanowych*. Aplikacja pozwala zgłaszać problemy, obsługiwać je po stronie technika oraz przeglądać podstawowe raporty, dzięki czemu dobrze odzwierciedla prosty system typu Service Desk. W projekcie została przygotowana relacyjna baza danych obejmująca ponad wymagane minimum pięciu tabel, połączonych zgodnie z założonymi relacjami, co było jednym z kluczowych wymagań omawianych na zajęciach.

8.2 Krytyczna analiza uzyskanych wyników

8.2.1 Analiza wybranego stosu technologicznego

Wybór Spring Boot 3.3.4 i Java 21 okazał się trafny dla projektu edukacyjnego. Spring Boot znacznie upraszcza konfigurację w porównaniu do czystego Springa, zapewniając dobrą równowagę między funkcjonalnością a prostotą użycia. Wbudowany serwer Tomcat i automatyczna konfiguracja profili (H2/PostgreSQL) umożliwiały szybkie przełączanie między środowiskami bez modyfikacji kodu aplikacji.

Frontend napisany w vanilla JavaScripcie (HTML5/CSS3) był wystarczający do celów demonstracyjnych, choć w przypadku bardziej skomplikowanego interfejsu użytkownika framework typu React lub Angular byłby bardziej efektywny. Dla tego projektu vanilla JavaScript okazał się wystarczający i pozwolił na łatwe debugowanie.

8.2.2 Co działało dobrze

- **Architektura trójwarstwowa** – wyraźny podział na warstwę prezentacji, logiki i danych ułatwił maintenance i testowanie poszczególnych komponentów.
- **JPA/Hibernate** – mapowanie obiektów na tabele bazy danych znacznie uprościło operacje CRUD bez pisania surowego SQL-a.
- **REST API** – interfejs niezależny od frontendu umożliwia łatwą zmianę lub dodanie nowych klientów (web, mobile, desktop).
- **H2 Database** – baza w pamięci pozwoliła na szybkie testowanie bez konieczności konfiguracji PostgreSQL w środowisku deweloperskim.
- **Maven** – zarządzanie zależnościami i budowa projektu były proste i powtarzalne.

8.2.3 Wyzwania i obszary do poprawy

- **Brak rzeczywistego uwierzytelniania** – Frontend używa symulacji ról zamiast komunikacji z endpointem `POST /api/login`. W produkcji wymagałoby to implementacji Spring Security z tokenami JWT lub sesja-based authentication.
- **Bezpieczeństwo haseł** – Hasła w tabeli `accounts` przechowywane są w postaci jawnej dla celów dydaktycznych. W rzeczywistej aplikacji wymaga to hashowania BCrypt.
- **Walidacja po stronie serwera** – Choć zaimplementowana na poziomie encji i DTO, brakuje bardziej zaawansowanej walidacji biznesowej (np. sprawdzanie SLA, automatyczne eskalowanie zgłoszeń).
- **Frontend bez frameworka** – Vanilla JavaScript powoduje powtórzenia kodu i trudność w zarządzaniu stanem dla bardziej złożonych interfejsów.

8.2.4 Porównanie z alternatywami

Projekt realizuje się mógł w różnych technologiach:

- **ASP.NET Core** – oferuje podobne możliwości do Spring Boot, z potencjalnie lepszą integracją Windows Server, ale wymaga licencji w środowisku produkcyjnym.
- **Django/Python** – szybsze prototypowanie, ale mniej wydajne dla aplikacji o wysokiej liczbie jednoczesnych użytkowników.
- **Node.js/Express** – dobrze sprawdza się dla aplikacji typu real-time, ale mniej niezawodny dla złożonych operacji bazodanowych.
- **Desktop (Access/C#)** – dla aplikacji wewnętrznych jest szybszy w tworzeniu, ale trudny do skalowania i wdrażania w sieci.

Wybór Spring Boot/Java okazał się optymalny dla tego typu projektu ze względu na stabilność, wydajność i dostępność dokumentacji.

8.2.5 Lekcje wyniesione z projektu

- Znaczenie poprawnego projektowania bazy danych na wczesnym etapie – zmiana struktury tabel po rozpoczęciu implementacji powoduje kaskadę zmian w kodzie i wymaga remapowania encji JPA.
- Rozdzielenie frontendu od backendu poprzez REST API daje elastyczność i umożliwia niezależne testowanie każdej warstwy.
- Użycie ORM (JPA/Hibernate) drastycznie skraca czas tworzenia aplikacji, choć wymaga zrozumienia podstaw mapowania relacyjno-obiektowego.
- Profilowanie środowiska (H2 dla dev, PostgreSQL dla prod) powinno być zaplanowane od początku projektu, aby uniknąć problemów kompatybilności.

8.3 Możliwe kierunki rozwoju

W przyszłości system może zostać rozszerzony o dodatkowe elementy, takie jak:

- logowanie i role z użyciem Spring Security,
- bezpieczne przechowywanie haseł (np. przy użyciu BCrypt),
- obsługa prostych zasad SLA dotyczących czasu reakcji,
- bardziej rozbudowane raporty oraz wykresy,
- eksport danych do PDF lub arkuszy kalkulacyjnych,
- szyfrowanie wybranych danych wrażliwych.

Takie rozszerzenia nie są wymagane w ramach projektu, ale pozwoliłyby przekształcić aplikację dydaktyczną w bardziej kompletny system, który można byłoby dalej rozwijać, np. jako temat pracy inżynierskiej.

Bibliografia

- Dokumentacja Spring Boot: `https://spring.io/projects/spring-boot`
- Dokumentacja H2 Database: `https://www.h2database.com/`
- Dokumentacja PostgreSQL: `https://www.postgresql.org/docs/`
- Przykłady i przewodniki do Spring Boot (tutoriale, artykuły, repozytoria GitHub).